The **Go net/http module** is the core package used to build **HTTP servers and clients** in Go. It's widely used in backend development and forms the base for frameworks like Gin, Echo, etc.

🚀 1. What is net/http?

- Standard library package: net/http
- Used for:
  - Creating **web servers**
  - Making **HTTP requests (clients)**
- Works on top of TCP

🧱 2. Basic HTTP Server (Hello World)

```
package main

import (
	"fmt"
	"net/http"
)

func helloHandler(w http.ResponseWriter, r *http.Request) {
	fmt.Fprintln(w, "Hello, World!")
}

func main() {
```

```
    http.HandleFunc("/hello", helloHandler)

    fmt.Println("Server running on http://localhost:8080")
    http.ListenAndServe(":8080", nil)
}
```

🔍 **Explanation**

- http.HandleFunc → maps URL to handler
- http.ResponseWriter → used to send response
- http.Request → contains request data
- http.ListenAndServe → starts server

🔁 **3. HTTP Handler Types**

Option 1: Function Handler

```
func handler(w http.ResponseWriter, r *http.Request)
```

Option 2: Struct Handler (Advanced)

```
type MyHandler struct{}

func (h MyHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    w.Write([]byte("From struct handler"))
}

func main() {
    http.Handle("/", MyHandler{})
```

```
	http.ListenAndServe(":8080", nil)
}
```

👉 Uses http.Handler interface

📦 4. Handling Different HTTP Methods

```
func userHandler(w http.ResponseWriter, r *http.Request) {
	switch r.Method {
	case http.MethodGet:
		fmt.Fprintln(w, "GET request")
	case http.MethodPost:
		fmt.Fprintln(w, "POST request")
	default:
		http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
	}
}
```

📜 5. Working with JSON (REST API)

Example: POST + JSON

```
package main

import (
	"encoding/json"
	"net/http"
```

```
)

type User struct {
	Name string `json:"name"`
	Age  int    `json:"age"`
}

func createUser(w http.ResponseWriter, r *http.Request) {
	var user User

	// Decode JSON request
	err := json.NewDecoder(r.Body).Decode(&user)
	if err != nil {
		http.Error(w, err.Error(), http.StatusBadRequest)
		return
	}

	// Response
	w.Header().Set("Content-Type", "application/json")
	json.NewEncoder(w).Encode(user)
}

func main() {
```

```
    http.HandleFunc("/users", createUser)
    http.ListenAndServe(":8080", nil)
}
```

🔗 6. Query Params & Path Params

Query Params

```
func queryHandler(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query().Get("name")
    w.Write([]byte("Hello " + name))
}
```

👉 URL: http://localhost:8080?name=Dhivya

Path Params (Manual Parsing)

```
func pathHandler(w http.ResponseWriter, r *http.Request) {
    id := r.URL.Path[len("/user/"):]
    w.Write([]byte("User ID: " + id))
}
```

📝 7. Middleware in Go

```
func loggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        println("Request:", r.URL.Path)
        next.ServeHTTP(w, r)
```

```
	})
}

func main() {
	mux := http.NewServeMux()
	mux.HandleFunc("/hello", helloHandler)

	http.ListenAndServe(":8080", loggingMiddleware(mux))
}
```

⚙️ 8. Custom Server Configuration

```
server := &http.Server{
	Addr:    ":8080",
	Handler: nil,
}

server.ListenAndServe()
```

🌐 9. HTTP Client (Calling APIs)

```
package main

import (
	"fmt"
```

```
	"io"
	"net/http"
)

func main() {
	resp, err := http.Get("https://jsonplaceholder.typicode.com/posts/1")
	if err != nil {
		panic(err)
	}
	defer resp.Body.Close()

	body, _ := io.ReadAll(resp.Body)
	fmt.Println(string(body))
}
```

🧠 10. Key Interfaces

Interface	Purpose
http.Handler	Handles HTTP requests
http.ResponseWriter	Writes response
http.Request	Incoming request

🏗️ 11. Production Structure (Recommended)

```
project/
```

```
│
├── main.go
├── handler/
│   └── user_handler.go
├── service/
│   └── user_service.go
├── repository/
│   └── user_repo.go
```

💡 12. Best Practices

- Always **close request body**
- Use **JSON encoding/decoding properly**
- Add **middleware (logging, auth)**
- Use **context for timeouts**
- Avoid global variables (use DI)

🔥 13. Mini CRUD Example (In-Memory)

```
var users = make(map[int]string)
var id = 1

func create(w http.ResponseWriter, r *http.Request) {
	name := r.URL.Query().Get("name")
	users[id] = name
```

```
	id++
	w.Write([]byte("User created"))
}

func getAll(w http.ResponseWriter, r *http.Request) {
	json.NewEncoder(w).Encode(users)
}

func main() {
	http.HandleFunc("/create", create)
	http.HandleFunc("/users", getAll)
	http.ListenAndServe(":8080", nil)
}
```

SQL integration:

Perfect 👍 — let's build a **production-grade REST API in Go using raw SQL (no GORM)** with **PostgreSQL**, while keeping the same clean architecture.

We'll use:

- database/sql (standard library)
- PostgreSQL driver: lib/pq
- Layered architecture (Handler → Service → Repository)

🏗️ 1. Project Structure

```
go-rest-api/
│
├── cmd/
│   └── main.go
├── config/
│   └── db.go
├── model/
│   └── user.go
├── repository/
│   └── user_repository.go
├── service/
│   └── user_service.go
├── handler/
│   └── user_handler.go
├── middleware/
│   └── logger.go
├── go.mod
```

⚙️ 2. Install Dependency

```
go mod init go-rest-api
go get github.com/lib/pq
```

🧱 3. Database Configuration (Raw SQL)

📁 **config/db.go**

```
package config

import (
	"database/sql"
	"log"

	_ "github.com/lib/pq"
)

var DB *sql.DB

func ConnectDB() {
	connStr :=
"postgres://postgres:postgres@localhost:5432/testdb?sslmode=disable"

	var err error
	DB, err = sql.Open("postgres", connStr)
	if err != nil {
		log.Fatal("DB open error:", err)
	}

	err = DB.Ping()
```

```
	if err != nil {
		log.Fatal("DB connection error:", err)
	}

	log.Println("Connected to PostgreSQL")
}
```

📄 4. Create Table (Run Once)

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name TEXT,
  email TEXT UNIQUE
);
```

📦 5. Model

📁 model/user.go

```
package model

type User struct {
	ID    int    `json:"id"`
	Name  string `json:"name"`
	Email string `json:"email"`
}
```

🗄️ 6. Repository Layer (RAW SQL)

📁 **repository/user_repository.go**

```
package repository

import (
	"database/sql"
	"go-rest-api/model"
)

type UserRepository interface {
	Create(user model.User) (model.User, error)
	FindAll() ([]model.User, error)
	FindByID(id int) (model.User, error)
}

type userRepository struct {
	db *sql.DB
}

func NewUserRepository(db *sql.DB) UserRepository {
	return &userRepository{db}
}
```

```
func (r *userRepository) Create(user model.User) (model.User, error) {
	query := `INSERT INTO users(name, email) VALUES($1, $2) RETURNING id`
	err := r.db.QueryRow(query, user.Name, user.Email).Scan(&user.ID)
	return user, err
}

func (r *userRepository) FindAll() ([]model.User, error) {
	rows, err := r.db.Query(`SELECT id, name, email FROM users`)
	if err != nil {
		return nil, err
	}
	defer rows.Close()

	var users []model.User
	for rows.Next() {
		var u model.User
		err := rows.Scan(&u.ID, &u.Name, &u.Email)
		if err != nil {
			return nil, err
		}
		users = append(users, u)
	}
```

```
	return users, nil
}

func (r *userRepository) FindByID(id int) (model.User, error) {
	var user model.User
	query := `SELECT id, name, email FROM users WHERE id=$1`
	err := r.db.QueryRow(query, id).Scan(&user.ID, &user.Name, &user.Email)
	return user, err
}
```

🧠 **7. Service Layer**

📁 **service/user_service.go**

```
package service

import (
	"go-rest-api/model"
	"go-rest-api/repository"
)

type UserService interface {
	CreateUser(user model.User) (model.User, error)
	GetUsers() ([]model.User, error)
	GetUser(id int) (model.User, error)
```

```
}

type userService struct {
	repo repository.UserRepository
}

func NewUserService(r repository.UserRepository) UserService {
	return &userService{r}
}

func (s *userService) CreateUser(user model.User) (model.User, error) {
	return s.repo.Create(user)
}

func (s *userService) GetUsers() ([]model.User, error) {
	return s.repo.FindAll()
}

func (s *userService) GetUser(id int) (model.User, error) {
	return s.repo.FindByID(id)
}
```

🌐 8. Handler Layer

📁 **handler/user_handler.go**

```go
package handler

import (
	"encoding/json"
	"go-rest-api/model"
	"go-rest-api/service"
	"net/http"
	"strconv"
	"strings"
)

type UserHandler struct {
	service service.UserService
}

func NewUserHandler(s service.UserService) *UserHandler {
	return &UserHandler{s}
}

func (h *UserHandler) CreateUser(w http.ResponseWriter, r *http.Request) {
	var user model.User
```

```
	err := json.NewDecoder(r.Body).Decode(&user)
	if err != nil {
		http.Error(w, err.Error(), http.StatusBadRequest)
		return
	}

	created, err := h.service.CreateUser(user)
	if err != nil {
		http.Error(w, err.Error(), http.StatusInternalServerError)
		return
	}

	json.NewEncoder(w).Encode(created)
}

func (h *UserHandler) GetUsers(w http.ResponseWriter, r *http.Request) {
	users, err := h.service.GetUsers()
	if err != nil {
		http.Error(w, err.Error(), http.StatusInternalServerError)
		return
	}
	json.NewEncoder(w).Encode(users)
}
```

```go
func (h *UserHandler) GetUser(w http.ResponseWriter, r *http.Request) {
	idStr := strings.TrimPrefix(r.URL.Path, "/users/")
	id, _ := strconv.Atoi(idStr)

	user, err := h.service.GetUser(id)
	if err != nil {
		http.Error(w, "User not found", http.StatusNotFound)
		return
	}

	json.NewEncoder(w).Encode(user)
}
```

📝 9. Middleware

📁 middleware/logger.go

```go
package middleware

import (
	"log"
	"net/http"
	"time"
)
```

```go
func Logger(next http.Handler) http.Handler {
	return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
		start := time.Now()
		next.ServeHTTP(w, r)
		log.Println(r.Method, r.URL.Path, time.Since(start))
	})
}
```

🚀 10. Main Entry

📁 cmd/main.go

```go
package main

import (
	"go-rest-api/config"
	"go-rest-api/handler"
	"go-rest-api/middleware"
	"go-rest-api/repository"
	"go-rest-api/service"
	"net/http"
)

func main() {
```

```
	config.ConnectDB()

	userRepo := repository.NewUserRepository(config.DB)
	userService := service.NewUserService(userRepo)
	userHandler := handler.NewUserHandler(userService)

	mux := http.NewServeMux()

	mux.HandleFunc("/users", func(w http.ResponseWriter, r *http.Request) {
		if r.Method == http.MethodPost {
			userHandler.CreateUser(w, r)
		} else if r.Method == http.MethodGet {
			userHandler.GetUsers(w, r)
		}
	})

	mux.HandleFunc("/users/", userHandler.GetUser)

	http.ListenAndServe(":8080", middleware.Logger(mux))
}
```

🧪 11. Testing

```
curl -X POST http://localhost:8080/users \
```

```
-H "Content-Type: application/json" \
-d '{"name":"Dhivya","email":"dhivya@test.com"}'

curl http://localhost:8080/users
curl http://localhost:8080/users/1
```

🔥 **Why Raw SQL (Important for You)**

✅ Full control over queries
✅ Better performance tuning
✅ No ORM overhead
✅ Closer to real DB behavior
✅ Required in many enterprise systems